

SERVICIO NACIONAL DE APRENDIZAJE - SENA
CENTRO DE FORMACION TECNOLOGICA | REGIONAL ANTIOQUIA

EVIDENCIA GA9-220501096-AA3-EV02
REPORTE DE PLAN DE PRUEBAS EJECUTADAS
Y TRAZABILIDAD DE CALIDAD

INFORME FINAL CONSOLIDADO DE ASEGURAMIENTO DE CALIDAD (QA)

Programa de Formacion: Analisis y Desarrollo de Software (ADSO)

Numero de Ficha: 3186645

Nombre del Aprendiz: Andres Mauricio Valencia Arango

Instructor de Seguimiento: Adornay Sanchez

Solucion de Software Evaluada: Sistema Web Camila Nails

Medellin, Colombia - 2026

1. INTRODUCCION

El presente documento constituye el informe de cierre y reporte técnico consolidado de las pruebas de software aplicadas a la solución web "Camila Nails". Como futuro tecnólogo en Análisis y Desarrollo de Software, comprendo que el desarrollo no concluye cuando se termina de escribir la última línea de código; por el contrario, es en la etapa de pruebas donde se demuestra si el sistema es verdaderamente confiable, estable y apto para ser utilizado por usuarios reales en un entorno productivo.

A lo largo de este reporte se documenta la trazabilidad completa del comportamiento del software, evaluando cada uno de los módulos principales que integran la aplicación: la autenticación de usuarios, el panel de administración, el registro y modificación de fichas técnicas de clientas, el control de accesos y la experiencia visual del usuario tanto en computadores como en teléfonos celulares. Asimismo, se describen las fallas que surgieron durante las jornadas de pruebas, explicando con detalle técnico qué originó cada error y cuál fue la solución implementada para resolverlo.

2. OBJETIVOS DEL REPORTE

2.1. Objetivo General

Presentar el reporte técnico consolidado de las pruebas planificadas y ejecutadas sobre el sistema Camila Nails, garantizando la trazabilidad entre los requerimientos funcionales, los casos de prueba y los resultados obtenidos, sustentando las acciones correctivas aplicadas para asegurar un producto de alta calidad.

2.2. Objetivos Especificos

- Comprobar el correcto funcionamiento de las operaciones CRUD (Crear, Leer, Actualizar y Eliminar) sobre la base de datos relacional MySQL.
- Evaluar la solidez y seguridad de los endpoints de la API REST desarrollada en Node.js frente a peticiones con datos incompletos o accesos no autorizados.
- Validar la ergonomía, legibilidad del texto en tema oscuro y la adaptabilidad responsiva del frontend en React.
- Dejar constancia formal de las métricas de efectividad y del estado final del sistema para el cierre de la competencia de calidad de software.

3. ALCANCE Y CONTEXTO DEL SISTEMA EVALUADO

El software "Camila Nails" es una aplicación web integral diseñada para sistematizar las operaciones diarias de un salón de estética y cuidado de uñas. La plataforma cuenta con una arquitectura desacoplada cliente-servidor:

1. Capa de Presentación (Frontend): Construida con React.js 18 y empaquetada mediante Vite. Brinda una interfaz moderna, con navegación fluida, soporte para tema claro y tema oscuro (Dark Mode), y un panel privado de administración.
2. Capa de Lógica del Negocio (Backend): Servidor estructurado con Node.js y el framework Express.js, encargado de exponer rutas RESTful protegidas, procesar la carga de imágenes y aplicar lógica de encriptación mediante bcryptjs.
3. Capa de Persistencia (Base de Datos): Motor MySQL Server con tablas relacionales normalizadas ("users" y "fichas"), comunicándose con el servidor mediante el controlador asíncrono mysql2.

4. AMBIENTE DE PRUEBAS UTILIZADO

Para garantizar que las pruebas se ejecutaran en un entorno controlado y reproducible, se configuró el siguiente laboratorio técnico en la estación de trabajo del analista:

- Estación de Trabajo: Portátil con procesador AMD Ryzen de 6 núcleos, 16 GB de memoria RAM, disco de estado sólido NVMe y sistema operativo Microsoft Windows 11 de 64 bits.
- Servidor de Aplicación: Entorno Node.js (v18.17.0) ejecutando Express en el puerto local 4000 (<http://localhost:4000>).
- Servidor de Base de Datos: MySQL Community Server 8.0.32, escuchando en el puerto TCP 3306, con almacenamiento de caracteres utf8mb4.
- Servidor de Desarrollo Frontend: Vite v5.0 sobre Node.js, sirviendo la aplicación React en <http://localhost:5173>.
- Herramienta de Pruebas de API: Postman v10.22, utilizada para la parametrización de peticiones HTTP, envío de payloads JSON y análisis de cabeceras de respuesta.
- Herramienta de Inspección de Interfaz: Google Chrome DevTools (versión 128) para simulación de dispositivos móviles (resoluciones de 360x740 y 412x915 píxeles) y auditoría de contraste de color.

5. MATRIZ DE TRAZABILIDAD (REQUERIMIENTO VS CASO DE PRUEBA)

La trazabilidad permite asegurar que no quede ningún requerimiento del sistema sin ser evaluado formalmente. A continuación se correlacionan los requerimientos funcionales del proyecto con los 15 casos de prueba ejecutados:

Req.	Nombre del Requerimiento	Casos Asociados	Estado Final
RF-01	Registro público de usuarios	CP-001, CP-002, CP-003	Aprobado (100%)
RF-02	Autenticación y seguridad de acceso	CP-004, CP-005	Aprobado (100%)
RF-03	Administración de usuarios (Listar/Editar/Borrar)	CP-006, CP-007, CP-008, CP-009	Aprobado (100%)
RF-04	Gestión de fichas técnicas de clientas (CRUD)	CP-010, CP-011, CP-012, CP-013	Aprobado (100%)
RNF-01	Accesibilidad visual y legibilidad en modo oscuro	CP-014	Aprobado (Corregido)
RNF-02	Diseño responsivo para dispositivos móviles	CP-015	Aprobado (Corregido)

6. REPORTE CONSOLIDADO DE RESULTADOS POR MODULOS

6.1. Modulo de Seguridad y Autenticacion

Este módulo fue sometido a pruebas de robustez mediante Postman. Se probó el registro con cargas útiles correctas (CP-001) obteniendo respuesta HTTP 200 OK y confirmando que la contraseña se almacene bajo un hash encriptado con bcryptjs y un factor de costo de 10 rondas. Asimismo, se pusieron a prueba los límites del servidor:

- En el caso CP-002, al omitir el correo electrónico, el backend interceptó la anomalía retornando de inmediato un código HTTP 400 Bad Request con el mensaje "Faltan campos", impidiendo que la base de datos reciba registros nulos.
- En el caso CP-003, al enviar un usuario con correo previamente registrado, el sistema respondió con código HTTP 409 Conflict ("Usuario o email ya existe"), respetando la restricción UNIQUE de MySQL.
- En el caso CP-005, el envío de credenciales con contraseña errada arrojó un código HTTP 401 Unauthorized, evitando accesos indebidos.

6.2. Modulo de Administracion de Usuarios y Roles

Desde el panel privado (/admin) y a través de los endpoints correspondientes, se evaluó la gestión de usuarios:

- Listado seguro (CP-006): La consulta GET a /api/users devuelve únicamente el identificador, nombre, usuario, correo y rol, omitiendo por diseño el campo "password" en la cláusula SELECT de SQL, garantizando privacidad.
- Modificación de rol (CP-007): Mediante peticiones PATCH a /api/users/:id/role se verificó el cambio dinámico de permisos entre "user" y "admin", actualizándose al instante en la interfaz.
- Edición y eliminación (CP-008 y CP-009): Se comprobó que el administrador pueda corregir nombres o correos en línea mediante peticiones PUT, y eliminar usuarios definitivamente mediante DELETE con confirmación previa del navegador.

6.3. Modulo de Fichas Clinicas de Clientes

Las fichas técnicas representan el corazón operativo del negocio para registrar el historial de las uñas de las clientas:

- Creación manual (CP-010): Se validó el formulario del panel de administración para registrar fichas directamente ingresando nombre, teléfono y servicio (ej. "Soft gel"), respondiendo con código 200 e insertando el registro en la tabla "fichas".
- Consulta histórica (CP-011): El endpoint GET /api/fichas recupera el listado ordenado cronológicamente para mantener el historial clínico.
- Edición y eliminación de fichas (CP-012 y CP-013): Las rutas PUT y DELETE en /api/fichas/:id operaron con total fidelidad, permitiendo corregir servicios o eliminar fichas obsoletas sin dejar datos huérfanos.

7. TRATAMIENTO DE INCIDENCIAS, DEFECTOS Y ACCIONES CORRECTIVAS

La verdadera trazabilidad de un proceso de pruebas radica en registrar con honestidad y rigor técnico qué falló, cómo se diagnosticó y de qué forma se corrigió para que el software alcanzara la madurez requerida. A continuación se exponen las dos incidencias principales encontradas durante el ciclo de vida del proyecto:

Incidencia 1: Ilegibilidad de texto por fondo blanco en foco (BUG-UI-01)

- Clasificación: Falla de Interfaz de Usuario / Defecto Visual (Severidad Media).
- Descubrimiento: Al realizar pruebas manuales en el formulario de creación de fichas bajo el tema oscuro (Dark Mode), se detectó que al hacer clic dentro de cualquier caja de texto para escribir, el texto se volvía totalmente invisible.
- Analisis Causa-Raiz: Se inspeccionó el archivo de estilos src/App.css con Chrome DevTools. Se descubrió que la pseudo-clase :focus tenía configurada la regla "background: rgba(255, 255, 255, 0.9);", la cual forzaba un fondo blanco brillante mientras el color del texto heredado en modo oscuro era blanco (#ffffff), ocasionando la pérdida total de contraste.
- Accion Correctiva Aplicada: Se eliminó dicha propiedad en src/App.css para que los campos preserven la variable semitransparente var(--field-bg) en su estado enfocado, manteniendo el contorno luminoso var(--accent) para orientar al usuario.
- Prueba de Regresion: Se repitió la prueba en modo oscuro; el texto ingresado se visualiza con absoluta nitidez y legibilidad.

Incidencia 2: Restricción rígida de ancho y desbordamiento móvil (BUG-UI-02)

- Clasificación: Falla de Responsividad / Diseno Adaptable (Severidad Baja).
- Descubrimiento: En pantallas de computadores de escritorio amplios (1080p o superior) la página se observaba como una columna angosta encerrada entre bordes con franjas negras vacías a los lados, mientras que en pantallas móviles estrechas se generaba un scroll horizontal indeseado.
- Analisis Causa-Raiz: En el archivo src/index.css, la etiqueta principal #root tenía un ancho forzado mediante "width: 1126px; border-inline: 1px solid var(--border);", impidiendo que la aplicación se adaptara al 100% de la ventana.
- Accion Correctiva Aplicada: Se sustituyó por "width: 100%; margin: 0 auto;", eliminando los bordes laterales y ajustando los contenedores hijos con max-width elásticos y paddings proporcionales en src/App.css.
- Prueba de Regresion: Se probó en resoluciones de 360px (móvil) y 1920px (escritorio), logrando una apariencia espaciosa, moderna y fluida de borde a borde.

8. METRICAS DE CALIDAD Y EVALUACION DE ESTABILIDAD

Al finalizar la jornada de ejecución de pruebas, se consolidaron los siguientes indicadores de desempeño y fiabilidad:

- Cobertura de Requerimientos: 100% de los requerimientos funcionales y no funcionales fueron cubiertos por los casos de prueba.
- Tasa de Éxito de Pruebas: 15 de 15 casos aprobados satisfactoriamente (100%).
- Eficiencia de Respuesta del Servidor: Las peticiones a la API REST promediaron tiempos de respuesta inferiores a 25 milisegundos en entorno local, demostrando la alta velocidad del motor Express y la conexión mediante mysql2.
- Tolerancia a Fallos: Ninguna petición con datos erróneos causó la detención abrupta (crasheo) del servidor Node.js, gracias a la adecuada implementación de bloques try/catch en todas las rutas asíncronas.

9. RECOMENDACIONES TECNICAS PARA EL DESPLIEGUE (DEPLOYMENT)

Con base en las lecciones aprendidas durante este proceso de evaluación, se sugieren las siguientes acciones preventivas para la futura etapa de puesta en producción:

1. Despliegue en la Nube: Alojarse el frontend estático de React en plataformas con CDN global (como Vercel o Netlify) y el backend de Node.js en servidores con reinicio automático (como Render o Railway).
2. Base de Datos en la Nube: Migrar la base de datos MySQL local a un servicio gestionado (Managed Database) con copias de seguridad automáticas diarias y certificados SSL/TLS activados.
3. Automatización de Pruebas: Implementar suites de pruebas automatizadas con Jest o Vitest para correr pruebas unitarias automáticas cada vez que se realice un commit en GitHub.

10. CONCLUSIONES

1. El diseño, ejecución y reporte de este plan de pruebas permitió transformar el software "Camila Nails" de un prototipo básico a un sistema de información robusto, seguro y visualmente agradable para clientas y administradores.
2. La trazabilidad mantenida a lo largo de este reporte demuestra que cada necesidad del negocio fue satisfecha técnicamente, y que los problemas encontrados durante el desarrollo se gestionaron mediante acciones correctivas documentadas y verificables.
3. Como aprendiz del programa ADSO, este ejercicio práctico reafirma la trascendencia de la disciplina de pruebas en la ingeniería de software: el testing no es un gasto de tiempo, sino la garantía de que el producto entregado sea motivo de orgullo profesional y confianza para el cliente.

Andres Mauricio Valencia Arango

Aprendiz / Analista de Pruebas y Calidad de Software

Programa: Analisis y Desarrollo de Software (ADSO) - Ficha 3186645

Servicio Nacional de Aprendizaje - SENA Regional Antioquia